

Language Modeling Overview

A **Language Model (LM)** is a statistical or neural method for determining the **probability of a sequence of words** occurring in a natural language. Essentially, a language model attempts to compute the probability $P(w_1, w_2, \dots, w_m)$ of seeing a sentence or a word sequence $w_1, \dots, w_m$.

The primary goal is to predict the **next word** w_i in a sequence, given the preceding words $w_1, \dots, w_{i-1}$, which is formulated as computing the conditional probability $P(w_i | w_1, \dots, w_{i-1})$.

Importance of Language Modeling

Language Modeling is foundational in Natural Language Processing (NLP) and is essential for:

- **Speech Recognition:** LMs help distinguish between homophones (e.g., "to," "too," "two") and choose the most probable word sequence given an acoustic input. 
- **Machine Translation:** They ensure the output translation is fluent and grammatically correct in the target language. 
- **Spell Check/Correction:** LMs can propose the most likely intended word given a typo. 
- **Generative AI/Text Generation:** The entire class of large language models (LLMs) like Gemini, ChatGPT, etc., are fundamentally advanced LMs, predicting the next token in a sequence to generate coherent text. 
- **Information Retrieval:** LMs can be used to rank documents based on how well they match a query. 

Types of Language Modeling

Language models can be broadly categorized based on their underlying approach:

1. Statistical Language Models (e.g., N-grams)

These models rely on counting the frequency of word sequences in a large corpus to estimate probabilities. 

- **N-gram Models:** Estimate the probability of the current word based on the previous $N - 1$ words (discussed in detail below). 
- **Unigram, Bigram, Trigram:** Specific N-gram models where $N = 1, 2$, and 3 respectively. 

2. Neural Language Models (e.g., RNNs, Transformers)

These use neural networks to learn the patterns and relationships in the data, offering better context understanding and generalization.

- **Recurrent Neural Networks (RNNs):** Use an internal memory (hidden state) to process sequences, but struggle with very long-range dependencies. 
- **Long Short-Term Memory (LSTM) / Gated Recurrent Units (GRU):** Variants of RNNs designed to better capture long-term dependencies. 
- **Transformer Models:** The current state-of-the-art. They use a mechanism called **Self-Attention** to weigh the importance of different words in the input, capturing long-range dependencies efficiently and in parallel. **LLMs are typically built on the Transformer architecture.** 
 



Language Model Markov Assumption and N-grams

Markov Assumption

A fundamental challenge in calculating $P(w_i | w_1, \dots, w_{i-1})$ is that the **context history** $w_1, \dots, w_{i-1}$ can be extremely long.

The **Markov Assumption** simplifies this calculation by asserting that the probability of the current word w_i depends **only** on the immediately preceding $N - 1$ words, rather than the entire history. ☺

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

N-gram Models

Based on this assumption, the **N-gram Model** is defined:

- An **N-gram** is a contiguous sequence of N items (words, letters, or tokens) from a given text. ☺
- The model estimates the conditional probability using the maximum likelihood estimate (MLE) from the training data:

$$P(w_i | w_{i-(N-1)}, \dots, w_{i-1}) = \frac{\text{Count}(w_{i-(N-1)}, \dots, w_{i-1}, w_i)}{\text{Count}(w_{i-(N-1)}, \dots, w_{i-1})}$$

Example: In a **Bigram** model ($N = 2$), we only look at the immediately preceding word (w_{i-1}): ☺

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

✖ The Curse of Dimensionality



The **curse of dimensionality** is a major problem for **N-gram models**, especially as N increases.

- **Problem:** As the size of the vocabulary ($|V|$) and the length of the N-gram (N) increase, the total number of possible N-gram sequences ($|V|^N$) grows **exponentially**.
- **Effect on LMs:**
 1. **Sparsity:** In any finite training corpus, most of the possible N-grams will **never be observed** (their count is zero).
 2. **Zero Probabilities:** This leads to a **zero probability** for unseen N-grams, making the model unable to generalize to new, valid sequences. For example, if the trigram "big furry dog" was never seen, its probability is zero, even if "big dog" and "furry dog" were common.
 3. **Memory:** Storing the counts and probabilities for all possible N-grams becomes computationally and memory-intensive.

Solution Attempts: Techniques like **Smoothing** (e.g., Laplace or Add-k smoothing) and **Interpolation** were developed to redistribute the probability mass from high-frequency counts to zero-count N-grams, mitigating the curse of dimensionality. **Neural Language Models** offer a more effective solution as they use **word embeddings** to represent words in a dense, continuous space, allowing them to generalize better even to unseen sequences. ⌚ ⌚

Language Model Implementation: Setup

The implementation requires basic data structures to store and count N-grams.

1. Setup and Libraries

You'll need fundamental Python libraries for data handling and tokenization:

- `collections.defaultdict`: Essential for easily storing counts and N-gram mappings without checking if a key exists first.
- `nltk` (**Natural Language Toolkit**): Useful for tokenization (splitting text into words) and sentence segmentation.
- `random`: Needed for the final step of randomly sampling text.

Python



```
import nltk
from collections import defaultdict
import random

# Use a specific N (e.g., Bigram, N=2)
N = 2
```

2. Reading Corpus

The first step is preparing the text data (the corpus) for training.

- **Read Text:** Load the raw text file.
- **Tokenization:** Convert the text into a sequence of tokens (words).
- **Preprocessing:** Convert to lowercase and handle punctuation.
- **Sentence Boundaries:** Add special markers (e.g., `<s>` for start, `</s>` for end) to handle N-grams at the beginning and end of sentences. This allows the model to learn the probability of starting and ending a sentence.

Python



```
def read_corpus(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        text = f.read().lower()

    # Simple tokenization by splitting into sentences and then words
    sentences = nltk.sent_tokenize(text)
    tokenized_corpus = []

    for sent in sentences:
        # Prepend (N-1) start tokens and append an end token
        tokens = ['<s>'] * (N - 1) + nltk.word_tokenize(sent) + ['</s>']
        tokenized_corpus.extend(tokens)

    return tokenized_corpus
```



Building the Model

The model is built by counting the frequency of N-grams.

3. N-grams Function

This function breaks the tokenized corpus into overlapping N-gram sequences.

Python



```
def get_ngrams(tokens, n):  
    """Generates a sequence of (context, word) tuples for training."""  
    ngrams = []  
    for i in range(len(tokens) - n + 1):  
        # The context is the first N-1 tokens  
        context = tuple(tokens[i : i + n - 1])  
        # The word to predict is the Nth token  
        word = tokens[i + n - 1]  
        ngrams.append((context, word))  
    return ngrams  
  
# Example (N=2 / Bigram):  
# Text: ['<s>', 'i', 'am', '</s>']  
# Ngrams: [ (('<s>',), 'i'), (('i',), 'am'), (('am',), '</s>') ]
```

4. Update Counts Function

This function trains the model by tallying the counts of all observed N-grams and their contexts.

Python



```
def update_counts(ngrams):
    # stores Count(context)
    context_counts = defaultdict(int)
    # stores Count(context, word)
    ngram_counts = defaultdict(lambda: defaultdict(int))

    for context, word in ngrams:
        ngram_counts[context][word] += 1
        context_counts[context] += 1

    return ngram_counts, context_counts
```

5. Probability Model Function

This step converts the raw counts into probabilities, usually by applying **Maximum Likelihood Estimation (MLE)**. Note that in a real implementation, this is where **smoothing** would be applied to handle zero counts.

$$P(w|\text{context}) = \frac{\text{Count}(\text{context}, w)}{\text{Count}(\text{context})}$$



```
def create_probability_model(ngram_counts, context_counts):  
    probability_model = defaultdict(lambda: defaultdict(float))  
  
    for context, next_words in ngram_counts.items():  
        total_context_count = context_counts[context]  
  
        for word, count in next_words.items():  
            # Calculate the MLE probability  
            # NOTE: Smoothing techniques (like Add-One) would be applied here  
            probability = count / total_context_count  
            probability_model[context][word] = probability  
  
    return probability_model
```

👉 Language Model Implementation: Sampling Text

The final step is to use the trained probability model to generate new text.

6. Sampling Text

The model generates text word-by-word, probabilistically selecting the next word based on the current context.

Python



```
def sample_text(model, start_token='<s>', length=20):
    # Initialize the current context with start tokens
    # Note: The context must be a tuple of size N-1
    current_context = tuple([start_token] * (N - 1))

    generated_text = []

    for _ in range(length):
        # 1. Get the probability distribution for the current context
        next_word_probs = model[current_context]

        if not next_word_probs:
            # If context is unseen (or if it leads to </s>), restart or break
            break

        # 2. Get choices (words) and weights (probabilities)
        choices = list(next_word_probs.keys())
        weights = list(next_word_probs.values())

        # 3. Randomly choose the next word using probability-weighted sampling
        next_word = random.choices(choices, weights=weights, k=1)[0]

        if next_word == '</s>':
            break # Stop if end-of-sentence token is generated

    generated_text.append(next_word)
```

Python



```
generated_text.append(next_word)

# 4. Update the context: shift the window
# Drop the first word and append the new word
current_context = current_context[1:] + (next_word,)

return " ".join(generated_text)
```

Putting It Together (Main Script)

To run the full model:

Python



```
# Assuming your text file is 'corpus.txt'
# 1. Read and tokenize
corpus_tokens = read_corpus('corpus.txt')

# 2. Get N-grams
all_ngrams = get_ngrams(corpus_tokens, N)

# 3. Update counts
ngram_counts, context_counts = update_counts(all_ngrams)

# 4. Create probability model
probability_model = create_probability_model(ngram_counts, context_counts)

# 5. Generate text
generated_sentence = sample_text(probability_model, length=30)
print(f"Generated Text (N={N}): \n{generated_sentence}")
```

This sequence provides a complete, working framework for implementing a basic N-gram language model for text generation.



The Markov Property and Markov Models

Markov Property

The **Markov Property** is the fundamental assumption that gives Markov Models their structure. It states that the probability of a system transitioning to any future state depends **only on the current state** and not on the sequence of events that preceded it (i.e., the past history is "forgotten").

- **Memorylessness:** The process is said to be memoryless.
- **Formal Definition (First-Order):** The probability of the next state S_t given the entire history is equal to the probability of S_t given only the immediate previous state S_{t-1} :

$$P(S_t | S_{t-1}, S_{t-2}, \dots, S_0) = P(S_t | S_{t-1})$$

The **N-gram Language Model** is a special case of a Markov Model, where the order $N - 1$ is the number of preceding words used as the "current state."

Markov Model (Markov Chain)

A **Markov Model** is a stochastic (random) model that describes a sequence of possible events (states) where the probability of moving to the next state depends only on the current state. The simplest Markov model is a **Markov Chain**, which is defined by:

1. A set of **States** (S).
2. An **Initial State Distribution** (π) defining the probability of starting in each state.
3. A **Transition Probability Matrix** (A) where each element a_{ij} is the probability of moving from state i to state j : $a_{ij} = P(S_t = j | S_{t-1} = i)$.

The probability of an entire sequence of states $S_1, S_2, \dots, S_T$ is calculated by multiplying the initial probability by the sequence of transition probabilities:

$$P(S_1, \dots, S_T) = P(S_1) \times P(S_2 | S_1) \times P(S_3 | S_2) \times \dots \times P(S_T | S_{T-1})$$

▴ Probability Smoothing and Log Probabilities

Probability Smoothing

In a language model trained by counting occurrences, a severe problem is **data sparsity**. If a specific sequence of words (an N-gram) has **zero** occurrences in the training corpus, its estimated probability is zero.

$$P(\text{unseen N-gram}) = 0$$

If this zero probability occurs anywhere in the calculation of a longer sequence (due to the product rule, see above), the entire sequence's probability becomes zero, deeming it **impossible**, which is unrealistic.

Smoothing is a technique used to **redistribute** some probability mass from seen events to unseen events, ensuring that all possible sequences have a non-zero probability.

- **Laplace (Add-One) Smoothing:** The simplest form. It adds 1 to all counts (both numerator and denominator counts), effectively giving a "virtual" count of one to every unseen event.

$$\hat{P}(w_i|w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + |\mathbf{V}|}$$

Where $|\mathbf{V}|$ is the vocabulary size.

Log Probabilities

Log probabilities ($\log P$) are used extensively in statistical and neural models for two main reasons:

1. **Computational Stability (Underflow):** Probabilities are values between 0 and 1. Multiplying many small probabilities together (as required for long sequences) results in an extremely small number that can quickly underflow the precision limits of standard computer arithmetic. By taking the logarithm, **multiplications become additions**:

$$\log P(\text{Sequence}) = \log(P_1 \times P_2 \times \dots \times P_T) = \log P_1 + \log P_2 + \dots + \log P_T$$

Adding negative numbers (since $\log(p)$ for $0 < p < 1$ is negative) is much more numerically stable.

2. **Model Comparison:** Using log probabilities, the model's confidence is represented by a number (typically negative). **Values closer to zero indicate a higher probability** (e.g., $\log(0.9) \approx -0.1$ is much better than $\log(0.01) \approx -4.6$). This allows for easy and robust comparison and scoring of different sequence candidates.

Building a Text Classifier

Markov Models (like N-gram LMs) or, more commonly, models based on similar probabilistic principles (like **Naive Bayes**), can be used for text classification. The goal is to classify a document D into one of several categories (classes) C_k .

Probabilistic Text Classification

Classification is based on finding the class C_k that has the maximum posterior probability given the document D :

$$\hat{C} = \arg \max_{C_k} P(C_k|D)$$

Using Naive Bayes (A Simplified Classifier)

The **Naive Bayes (NB) Classifier** is widely used for text classification and is conceptually similar to a Markov Model but with an even stronger independence assumption. It uses Bayes' Theorem:

$$P(C_k|D) \propto P(C_k) \times P(D|C_k)$$

Where $P(D|C_k)$ is the likelihood of the document given the class. To make the computation tractable, NB makes the "**Naive**" **Assumption**: all words in the document are conditionally independent of each other given the class.

$$P(D|C_k) = P(\text{word}_1, \dots, \text{word}_m|C_k) \approx \prod_{i=1}^m P(\text{word}_i|C_k)$$

Steps to Build a Naive Bayes Text Classifier:

1. **Training:** Calculate the probabilities from the training data:
 - **Prior Probability** $P(C_k)$: The fraction of documents belonging to class C_k .
 - **Likelihood** $P(\text{word}_i|C_k)$: The probability of word word_i appearing in a document of class C_k (estimated using counts, often with smoothing to prevent zero probabilities).
2. **Classification:** For a new document D , calculate $P(C_k) \times \prod P(\text{word}_i|C_k)$ for every class C_k .
3. **Prediction:** The class $\hat{C}$ with the highest final probability is the predicted class.

While Naive Bayes assumes independence, **Markov Models** (specifically N-gram LMs) explicitly capture **local dependencies** (word word_i depends on word_{i-1}), which makes them better suited for tasks like text generation, but they are generally less efficient and more prone to sparsity than NB for simple document classification.

Article Spinning: Problem Description

Article Spinning is the process of generating a new piece of text that conveys the same meaning as a source text but is substantially different in word choice and phrasing.

Problem Goal

The primary goal of article spinning is to **create unique content quickly** from existing articles. This was historically done to:

- **Avoid Plagiarism Detection:** Generating text that passes basic checks for identical word sequences.
- **Search Engine Optimization (SEO):** Creating many variations of an article to be indexed by search engines, potentially boosting search rankings.

The Challenge

The core challenge is balancing two competing requirements:

1. **High Uniqueness:** The output must be structurally and lexically different from the input.
2. **High Coherence and Grammaticality:** The output text must be fluent, make sense, and preserve the original article's core meaning.

Simple spinning often leads to poor-quality, unnatural, or nonsensical text.

N-Gram Approach for Article Spinning

The N-Gram approach is a **statistical method** that can be used to generate new text that is locally fluent, based on the probability distributions learned from a corpus of text.

Mechanism

The N-Gram Language Model is trained on a large corpus of text (or the source article itself). This model computes the probability of a word given the previous $N - 1$ words.

1. **Training:** The model counts the occurrences of all N -word sequences (N-grams).
2. **Prediction/Generation:** When generating a new word, the model looks at the last $N - 1$ words (the context) and uses the learned probabilities to choose the next word.

How it Spins

Instead of deterministically choosing the *most* probable word, the spinning process uses the N-gram model to introduce variability:

- **Probabilistic Choice:** The model considers all words that have appeared after the current context ($N - 1$ words) and selects the next word randomly, weighted by their calculated probabilities.
- **Example (Bigram, $N = 2$):** If the current word is "The", and the model has seen:
 - "The **cat**" (60% of the time)
 - "The **dog**" (30% of the time)
 - "The **quick**" (10% of the time) The spinning algorithm will choose "cat" 60% of the time, "dog" 30% of the time, and "quick" 10% of the time. This random, probability-weighted selection ensures variety while maintaining local grammatical coherence.
- **Trade-off:** While this guarantees local fluency (the transition from w_{i-1} to w_i is probable), it often fails to maintain **long-range coherence** and meaning, resulting in text that quickly drifts from the original topic.

Cipher Decryption and Substitution Ciphers

Ciphers

A **cipher** is an algorithm for performing encryption and decryption—a series of well-defined steps that can be followed as a procedure.

Substitution Cipher

A **substitution cipher** is one of the simplest types of ciphers. The core mechanism is to replace every unit of plaintext (usually a single letter) with a fixed unit of ciphertext.

- **Key:** A mapping (or permutation) of the alphabet. For example, the key might map 'A' $\rightarrow$ 'X', 'B' $\rightarrow$ 'Q', and so on.
- **Decryption Problem:** Given a ciphertext, the goal is to find the correct 26-letter permutation (the key) that maps the ciphertext letters back to the plaintext letters, making the resulting text readable. The total number of possible keys is $26! \approx 4 \times 10^{26}$, making brute-force search impossible.

Language Modeling for Decryption

Since a successful decryption will produce coherent, grammatically correct text, a **Language Model (LM)** can be used as a **fitness function** to evaluate the quality of a proposed decryption key.

Bigrams

To implement an LM for scoring, the **Bigram Language Model** is typically used due to its balance of simplicity and effectiveness.

- A **Bigram LM** calculates the probability of a word w_i given only the previous word w_{i-1} , i.e., $P(w_i|w_{i-1})$.
- For substitution ciphers, the LM is applied at the **character level**, calculating the probability of a character c_i given the previous character c_{i-1} , i.e., $P(c_i|c_{i-1})$.
- The Bigram probability matrix $P(c_i|c_{i-1})$ is trained on a massive corpus of typical plaintext (e.g., English text). It tells us how common letter sequences are (e.g., 'TH' is very high, 'XQ' is very low).

Maximum Likelihood and Log-Likelihood

To score a potential decryption key, we calculate the **likelihood** of the resulting plaintext C occurring based on the Bigram LM.

- **Likelihood:** The probability of the entire decrypted sequence (plaintext) is calculated by multiplying the conditional probabilities of all its bigrams (assuming the Markov Property):

$$P(C) = P(c_1) \times P(c_2|c_1) \times P(c_3|c_2) \times \dots \times P(c_m|c_{m-1})$$

A **better key** results in a **higher** $P(C)$.

- **Log-Likelihood:** To maintain numerical stability and prevent underflow (multiplying many small probabilities), we use the **log-likelihood** (or log probability). Log-likelihood transforms multiplication into summation, which is computationally easier and more robust:

$$\log P(C) = \log P(c_1) + \sum_{i=2}^m \log P(c_i | c_{i-1})$$

The goal of the decryption algorithm is to find the key that **maximizes the log-likelihood** of the resulting plaintext.

Genetic Algorithms (GA)

A **Genetic Algorithm (GA)** is a powerful metaheuristic inspired by the process of natural selection. It is used to find optimal solutions to search problems where the search space is too large for exhaustive search (like the $26!$ keys).

Core Components

The GA simulates evolution by repeatedly generating and evaluating a **population** of candidate solutions (keys).

1. **Individual/Chromosome:** In this context, an **individual** is a complete 26-letter substitution **key** (a permutation of the alphabet). This is the proposed solution.
2. **Fitness Function:** The **Log-Likelihood** calculated by the Bigram Language Model is the fitness function. It quantifies how "English-like" (i.e., how good) a potential decryption key is. The higher the log-likelihood, the fitter the key.
3. **Selection:** Better-fitting individuals (keys resulting in higher log-likelihood) are chosen to survive and breed.
4. **Crossover (Recombination):** Two "parent" keys are combined to create "child" keys. Since a key must be a unique permutation, crossover involves swapping portions of the mapping while ensuring no letter is mapped to itself or to another letter twice.
5. **Mutation:** Random, small changes are introduced to a child key (e.g., swapping two randomly chosen mappings) to introduce diversity and prevent the algorithm from getting stuck in a local maximum.

The Decryption Process

1. **Initialization:** Generate a large, random initial population of keys (e.g., 100 random permutations).
2. **Evaluation:** Calculate the **log-likelihood** for the plaintext produced by each key using the Bigram LM.
3. **Loop:** Repeat for many generations:
 - **Selection:** Select the best-performing keys based on their log-likelihood score.
 - **Crossover:** Generate new keys by crossing over the selected parents.
 - **Mutation:** Randomly mutate a small percentage of the new keys.
 - **Replacement:** Replace the old, low-fitness population with the new generation.
4. **Termination:** Stop when the maximum log-likelihood score stops improving or a maximum number of generations is reached. The best key found is the decryption key.

The GA efficiently explores the vast key space by prioritizing and refining the promising candidates, using the language model score to guide the search toward genuinely readable solutions.